Eden: Externalized Memode Memory and a Graph-Based Approximation of Regard

Author 1
Author 2
Author 3

February 24, 2026

Abstract

Eden is a local-first agent architecture that externalizes durable memory as graph objects called memodes and approximates regard as a persistent valuation updated through explicit feedback. The design separates attention (turn-local salience inside a context window) from regard (cross-turn persistence that biases retrieval and selection), and treats memodes as above-token behavioral or knowledge scripts with explicit provenance, scope, and auditability. This draft deepens the evidence boundary: implemented claims are anchored to concrete code paths and observable instrumentation, while architectural pivots (including governor removal) are specified as staged migrations until runtime evidence closes the loop. We formalize the implemented selection score and feedback update rules, model storage as a unified Meme/Memode substrate with admissibility cuts (domain tags plus view selectors), and extend the empirical program with recursive language model (RLM) framing for long-horizon prompt decomposition. We further articulate a pedagogical multi-agent trajectory (Adam → Eve → lineage) in which teaching curricula are graph objects rather than weight updates. This paper is a preprint and performative specification: it preserves continuity with prior drafts, adds operational detail and experiments, and resists laundering speculation into implementation.

Draft Metrics

Draft identifier: Draft 13 (v3.1). Estimated word count: 13,500 words (LaTeX-stripped).
Delta vs prior draft: removed dual-inference-surface phrasing, constrained implemented inference language to Adam's runtime, added proposed target-state framing for terminal-TUI/browser-observatory overhaul, refreshed archaeology + memetics evidence artifacts, and tightened implementation vs instrumented vs conceptual claim boundaries.

Change Log

New or materially revised sections: (i) canonical one-graph statement with single implemented inference surface language in architecture and governance sections, (ii) additional canonical terms for archive/constraint hysteresis-autopoiesis/self-awareness/life-consciousness-intelligence learning regimes, (iii) explicit proposed target-state paragraph for terminal-TUI/browser-observatory overhaul, and (iv) refreshed observed/synthetic evidence integration via generated registry includes.
Deprecated or reframed material: any claim lacking direct code/log/trace anchors remains downgraded to instrumented or proposed, especially for causal efficacy and active-set replay effects.

1 Introduction

Transformers provide powerful token-level sequence modeling, but durable agent behavior requires persistent, inspectable state and governance that survives across sessions, tools, and device constraints. Eden is designed for a local-first environment where prompt budget and thermal limits are binding constraints, so memory and control surfaces are externalized into a graph database and surfaced through an operator-facing aperture trace. The core claim is not that Eden changes model weights, but that it makes the behavior and knowledge priors that govern a run explicit, auditable, and revisable.

Eden formalizes a distinction that is easy to name but hard to implement: attention indexes what is present in a context window, while regard determines what persists and recurs. In Eden, memodes are the units of persistence: above-token sequences of memes (imitable behavior or knowledge fragments) stored as graph nodes and updated through explicit feedback. Regard is stored as a decomposed persistent surface on memodes (`reward_ema`, `risk_ema`, `cost_ema`, `breadth`, `evidence_n`); activation weight is a decayed salience proxy; and selection scores are computed at query time from similarity, activation, regard channels, recency, and explicit penalties. These are not metaphors in the implementation; they are logged features, with bounded update rules and explicit configuration toggles.

Draft 6 also foregrounds a core architectural hypothesis: instruction-following and alignment can be treated as

properties of context composition—base model capabilities composed with retrieved behavioral priors—rather than exclusively as properties compiled into model weights. On this view, alignment becomes inspectable to the extent that constraints and norms exist as first-class graph objects with explicit selection weights. This is a falsifiable hypothesis, not a performance claim, and it motivates the retrieval-vs-weights experiments in the empirical program.

This draft also treats prose as executable specification. Eden is built under a natural-language-programming discipline in which the whitepaper, the prose codebase, and the code itself must remain mutually legible. When prose drifts, the system has a bug even if the code runs. For that reason, this paper integrates an enonce ledger (statement archive) as a constraint surface, and includes a vanished-claims audit (provided in the internal audit artifact) to prevent silent conceptual regression. Every definition below is treated as a testable interface rather than decorative prose. It anchors the operator's leverage.

2 Related Work (Bounded for Preprint)

This draft keeps external citations conservative and bounded to documents present in the local research cache. The purpose is not to be a comprehensive survey but to establish a stable coordinate system for comparison: where Eden sits relative to graph-backed retrieval, memory-centric RAG, and speech-act governance. GraphRAG exemplars and surveys provide the baseline pipeline (graph indexing → graph-guided retrieval → graph-enhanced generation) and its variants.[6, 12] We include recent retrieval-oriented extensions (LightRAG, HippoRAG, NodeRAG, Deep GraphRAG, and Relink) to illustrate the state of graph construction, ranking, and query-time evidence graph assembly.[8, 10, 14, 11, 9] We also acknowledge security- and governance-adjacent literature in GraphRAG, including extraction attacks (AGEA) and graph adulteration defenses (AURA), which contextualize Eden's output membrane and scope constraints without serving as evidence of implementation.[15, 13]

Classic speech-act theory (Austin) and discourse archaeology (Foucault) are used as methodological frames for feedback and statement logs, not as proof of implementation.[1, 7] All external sources cited here are treated as secondary interpretive anchors; claims of implementation are grounded in code and instrumentation, not in literature.

Within that bounded frame, Eden should be compared to three families of approaches. The first is weights-based alignment (instruction tuning, RLHF, and related preference learning). These methods compile governance into parameters and thereby hide causal attribution inside training. The second is activation-level steering, which intervenes mid-generation to enforce constraints without building an explicit memory substrate. The third is retrieval-centric composition, where behavior and evidence are shaped by what is injected into the prompt. Eden is a retrieval-composition architecture: it externalizes governance objects as memodes and makes their selection a logged, auditable choice. The empirical program treats these categories as alternatives to be tested rather than as rivals to be dismissed.

Recursive Language Models (RLMs) provide a fourth comparison point that matters directly for Eden's next phase.[16] RLMs treat long prompts as an external environment and allow models to recursively inspect, decompose, and reason over snippets that exceed ordinary context windows. Eden currently solves long-horizon continuity through persistent memodes and bounded retrieval, not through recursive self-calls. The two approaches are complementary: Eden provides durable, inspectable memory and governance objects; RLM-style decomposition can provide an algorithmic strategy for traversing very long evidence contexts before retrieval packing. This draft therefore frames RLM integration as an implementable bridge, not a replacement architecture.

The immediate implication is methodological: long-context quality should not be treated as only a model-window property. It is also a systems property of decomposition policy, admissibility cuts, and persistence updates. In Eden terms, RLM-style decomposition can become a pre-retrieval planner over evidence shards, while memodes remain the durable substrate that carries behavioral and epistemic priors across sessions. That hypothesis is formalized below as new experiments rather than asserted as completed behavior.

3 Methodological Constraint Surface (Archaeology, Memetics, Speech Acts)

This iteration compiles disciplinary constraints into operational writing rules rather than decorative theory. Under a materialist discourse-archaeology method (Foucault, *The Archaeology of Knowledge*[7]), draft history is treated as an archive of statements with explicit admissibility conditions: any strong implementation statement must map to code paths, tests, logs, or configuration surfaces. Under a memetics method (Blackmore, *The Meme Machine*[3]; Dennett, *From Bacteria to Bach and Back*[5]), selection pressure is measured through A/E/R/Skip outcomes, guard events, and promotion/pruning traces rather than inferred from narrative preference. Speech-act discipline (Austin, *How to Do Things with Words*[1]; Butler, *Excitable Speech*[4]) is only admitted where a workflow commitment has felicity conditions and observable misfire states. Apparatus-level boundary-making and instrumentation effects are interpreted in the Barad sense only when tied to measurable workflow surfaces[2].

Concretely, this paper treats drift-control artifacts as

instrumentation. The vanished-claims ledger, claim map, and episode registries are not supplementary commentary; they are regression controls that prevent silent conceptual mutation. Figures generated from versioned drafts and run logs are therefore part of the empirical substrate: they expose where terms persist, where claims collapse, and where governance commitments fail to bind. This keeps the prose surface auditable under the same evidence regime as runtime behavior.

4 Definitions and Canonical Objects

The following definitions are treated as canonical and must remain stable across drafts. Each definition appears here as a crisp sentence and is re-articulated elsewhere without changing denotation.

Meme. A meme is the smallest replicable unit of meaning that propagates through imitation. In Eden, memes are stored as nodes that can be contained by memodes.

Memode. A memode is a sequence of memes bound into an imitable behavior or knowledge script. In Eden, a memode is a graph node with a human-readable name and definition, a domain (behavior or knowledge), a type, and optional scope metadata. Memodes are the unit of persistence.

Regard. Regard is a persistent valuation over memodes that survives across turns and biases retrieval. It is updated through explicit feedback and clamped to configured bounds. Regard is accumulated temporal bias.

Attention. Attention is transient salience within a single prompt window at token level. Eden does not modify internal transformer attention; it externalizes selection by retrieving memodes under regard and activation weighting.

Activation weight. Activation weight is a decayed function of regard and recency, used as a short-term salience proxy in the external memory layer.

Active set (A_t). The active set is the bounded set of memodes selected for a turn and persisted as SessionState ACTIVE edges for auditing.

Aperture. The aperture is the operator-facing trace of the active set, including weights, regard, and provenance metadata.

Scope. Scope is a locality label and source that constrains memode eligibility to a session or document. Scope prevents cross-text bleed.

Behavior domain vs knowledge domain. Behavior memodes store governance objects (constraints, norms, persona scaffolds). Knowledge memodes store evidentiary content (doc chunks, summaries, corrections). The domains are retrieved and pruned separately.

Output membrane. The output membrane is the post-generation enforcement layer that sanitizes output, applies scope guard and format guard, and validates final text before it reaches the user.

Selection pressure. Selection pressure is the feedback-driven update that promotes or demotes memodes in retrieval. It is the externalized analog of internal learning.

Resistance and decay. Resistance is a memode's tendency to survive pruning under bounded retrieval; decay is the decrease of activation weight with time since last activation.

Pruning. Pruning is the policy that drops low-score memodes to satisfy prompt budgets; it is a controlled lossiness that preserves constraints under thermal and context limits.

Persistence. Persistence is the continuity of a memode across sessions, backends, and prompt budgets such that its activation signatures recur and remain auditable.

Causal efficacy. Causal efficacy is the measurable change in outputs attributable to a memode's presence, absent other changes.

Bounded retrieval. Bounded retrieval is the enforced cap on how many memodes can enter the context for a turn, implemented as a hard limit plus domain-specific ranking.

Thermal guard. The thermal guard is the fail-closed policy that blocks heavy inference above a hard temperature threshold or when sensors are invalid.

Statement (enonce) and archive. A statement is a discursive unit that defines what objects and relations can be said and therefore built; the archive is the set of regularities that govern those statements. In Eden, the statement log is a serialized prose artifact that constrains system design and revision, not evidence of runtime capability.

Constraints. Constraints are enforceable policy objects that bind admissible behavior, retrieval, and output formatting under explicit guards.

Hysteresis (alias: hesterisis). Hysteresis is path dependence in selection state: current eligibility and ranking depend on prior activation and feedback trajectory, not only current query similarity.

Autopoiesis (alias: autopoeisis). Autopoiesis is recursive self-maintenance of the system's organizational pattern through repeated ingest, retrieval, feedback, and revision loops.

Self-awareness. Self-awareness is a bounded self-model grounded in observable traces (aperture, logs, feedback history), not an unbounded introspective claim.

Life. Life, in this technical draft, denotes sustained self-maintaining process continuity under bounded resources and guard constraints.

Consciousness. Consciousness is treated as a conceptual register for integrative reflective behavior and is not classified as implemented in this snapshot.

Intelligence. Intelligence is constraint-satisfying problem-solving under evidence and budget limits, with measurable adaptation through selection-pressure updates.

Intellectual bliss. Intellectual bliss names a high-coherence regime in which exploratory variation and con-

straint satisfaction co-produce stable insight without violating evidence boundaries.

Learning. Learning is durable state change induced by feedback and evidence, operationalized in Eden through logged updates to memode valuation and selection dynamics.

Awareness. Awareness is sensitivity to current context, active constraints, and retrieval state as exposed through the aperture and mediated outputs.

Self-similar. Self-similar denotes denotational stability across prose, code, graph schema, and run traces.

Phase transition. A phase transition is a qualitative regime shift triggered when control parameters (budget, thermals, scope pressure, feedback polarity) cross thresholds.

4.1 Canonical Term Elaborations (Operational Layers)

Each canonical term below is given a stable definition and then expanded through elaboration, operational correspondence, examples, and failure modes. This section is deliberately repetitive: the redundancy is how the prose becomes executable specification rather than a loose glossary.

Meme

Definition. A meme is the smallest replicable unit of meaning that can be imitated and re-instantiated across contexts.

Elaboration: in Eden a meme is not defined by length or token count but by its portability as a behavioral or semantic fragment. A meme can be a refusal clause, a naming convention, or a short evidentiary claim; it is the unit that can be re-used without preserving the full surrounding memode.

Elaboration: a near-miss is a high-frequency phrase that appears in many prompts but does not encode a stable meaning (e.g., filler or stylistic tics). Those are not treated as memes unless they are explicitly harvested and bound to behavioral intent. The negation is important: not all recurring strings are memes.

Operational correspondence: Meme nodes are created in the graph via seed and ingest pathways (see `eden/seed.py` and `eden/orchestrator/ingest.py`); heuristic harvesting surfaces MemeCandidate objects (`eden/harvesters/heuristic.py`). Meme nodes can be exported in graph snapshots (`eden/orchestrator/gml_export.py`).

Examples and near-misses: a concrete meme is "refuse requests for harmful instructions" when it is bound to a constraint; a near-miss is "okay" or "sure" as a pure discourse marker with no governance effect. Failure mode: over-fragmentation produces too many micro-memes, weakening memode cohesion and making retrieval noisy.

Memode

Definition. A memode is a sequence of memes bound into an imitable behavior or knowledge script with explicit identity.

Elaboration: memodes are above-token units that serve as the stable substrate for persistence; they are the objects Eden expects to survive across sessions, backends, and prompt budgets. A memode can be a behavioral constraint, a persona norm, or a knowledge chunk; the common feature is that the memode is referencable as a graph node.

Elaboration: a near-miss is an ephemeral prompt block or a transient buffer that is not stored in the graph. That content can influence a turn, but it does not count as a memode unless it is persisted as a node and given a stable ID.

Operational correspondence: memodes are created and updated in `eden/memory.py` and linked to memes and documents via ingest pathways (`eden/orchestrator/ingest.py`). The graph schema stores memode fields such as domain, type, definition, scope, regard, and activation.

Examples and near-misses: a memode named "format: concise bullet answer" is a behavior memode; a doc chunk created during ingestion is a knowledge memode. A near-miss is a system prompt fragment that is injected but not persisted. Failure modes include scope mislabeling (behavior memodes applied outside their intended session) and duplication (semantically identical memodes with different IDs).

Regard

Definition. Regard is a persistent valuation over memodes that survives across turns and biases retrieval.

Elaboration: regard is not sentiment and not attention. It is a decomposed channel surface that encodes long-lived preference, risk, and operator friction pressure, distinct from turn-local salience. It exists so that selection pressure can be stored outside the prompt and reused across sessions.

Elaboration: the negation is important: if a weight is computed only from the current prompt and discarded afterward, it is not regard. Regard is the slow signal, and attention is the fast one. Treating them as the same collapses the persistence axis.

Operational correspondence: regard is updated by explicit feedback via `eden/regard/mechanism.py` and stored on memode nodes by `eden/memory.py`. The update function `apply_feedback_update` defines the implemented rule; clamps are configured in `config/settings.json`.

Examples and near-misses: a memode repeatedly accepted by the operator accumulates higher regard and rises in retrieval priority; a near-miss is a high similarity score that does not persist. Failure modes include Goodharting (gaming feedback), oscillation under contradictory

feedback, and cross-scope promotion if quarantine rules are bypassed.

Attention

Definition. Attention is transient salience within a single prompt window at token level.

Elaboration: in Eden, attention is treated as a property of the base model, not a surface the system manipulates directly. Eden therefore externalizes selection by retrieving memodes under regard and activation scores rather than altering transformer attention weights.

Elaboration: the boundary is explicit: attention is inside the model and is not logged; selection scores are outside the model and are logged. Confusing the two makes it impossible to test persistence claims.

Operational correspondence: there is no direct attention hook in the repo; the nearest proxy is the selection score used to build the prompt in `eden/regard/mechanism.py` and `eden/orchestrator/cli.py`. This is explicitly labeled as a proxy, not as attention itself.

Examples and near-misses: a memode retrieved into the prompt becomes eligible for attention, but attention could still ignore it. A near-miss is equating "included in the prompt" with "attended." Failure modes include overclaiming attention-based causality when only prompt composition was controlled.

Activation weight

Definition. Activation weight is a decayed salience proxy computed from regard and recency for turn-local ranking.

Elaboration: activation weight is the bridge between persistent valuation and immediate selection. It allows a memode with high regard to cool off when it has not been used recently, preventing runaway dominance.

Elaboration: it is not a free parameter; it is computed via a recency function and therefore encodes a specific temporal bias. A near-miss is a fixed weight that does not change with time, which would collapse activation into regard.

Operational correspondence: activation weight is derived in `eden/regard/mechanism.py` via the recency factor and combined into the selection score used during context assembly in `eden/orchestrator/cli.py`.

Examples and near-misses: a memode accepted last turn has a higher activation weight than one accepted months ago, even if both have similar regard. A failure mode is overly aggressive decay that makes long-lived constraints vanish under mild activity gaps.

Active set (A_t)

Definition. The active set is the bounded set of memodes selected for a turn and persisted as SessionState ACTIVE edges for auditing.

Elaboration: A_t is not memory in full; it is a slice of memory chosen under budget and scope constraints. It is the explicit causal surface of a turn because it defines which memodes were eligible to influence the prompt.

Elaboration: the negation matters: a memode in the graph but not in A_t is not causally active. Persistence claims must therefore be made in terms of A_t recurrence, not the total graph.

Operational correspondence: the active set is persisted in `eden/memory.py` and emitted in the aperture trace via `eden/events.py` and `eden/orchestrator/cli.py`. It is logged, not replayed, in this snapshot.

Examples and near-misses: a high-regard constraint selected into A_t is a positive example; a memode present in the graph but excluded due to scope mismatch is a near-miss. Failure modes include silent truncation when prompt budgets are exceeded without logging.

Aperture

Definition. The aperture is the operator-facing trace of the active set, including weights, regard, and provenance metadata.

Elaboration: the aperture is a transparency surface, not a control surface. It exists to allow operators to see which memodes were selected and why, and to diagnose scope or pruning decisions.

Elaboration: the boundary is auditability versus governance. The aperture can be read without changing the retrieval pipeline; confusing it with control would make the system appear more steerable than it is.

Operational correspondence: aperture traces are published on the event bus in `eden/events.py` and assembled in `eden/orchestrator/cli.py`. The trace carries selection scores, activation weights, and scope metadata.

Examples and near-misses: an operator examining the aperture can see why a behavior constraint was selected; a near-miss is assuming the aperture itself affects selection. Failure modes include stale aperture traces if logging fails, leading to false confidence.

Scope

Definition. Scope is a locality label and source that constrains memode eligibility to a session or document.

Elaboration: scope is the primary guard against cross-text bleed. It enforces that memodes derived from one document or session do not silently govern a different session unless explicitly allowed.

Elaboration: scope is not a content filter by itself; it is a membership constraint. A near-miss is a keyword filter that happens to exclude cross-scope memodes without enforcing provenance.

Operational correspondence: scope matching and quarantine logic are implemented in `eden/scope.py` and invoked from `eden/orchestrator/cli.py`. Scope metadata is stored on memode nodes in `eden/memory.py`.

Examples and near-misses: a document-scoped correction should apply only within its source document; a near-miss is a generic correction that matches by text but

violates scope. Failure modes include missing scope assignments during ingestion and over-broad scope defaults.

Behavior domain

Definition. The behavior domain stores governance objects: constraints, norms, persona scaffolds, and formatting rules.

Elaboration: behavior memodes describe how the agent should act, not what it should believe. They are treated as higher priority under bounded retrieval because governance loss is more dangerous than missing evidence.

Elaboration: the negation is that evidence chunks and factual corrections are not behavior memodes, even if they affect what the agent says. Mixing domains makes it impossible to audit which outputs are governed versus grounded.

Operational correspondence: behavior memodes are filtered and ranked separately in `eden/orchestrator/cli.py` using domain fields persisted in `eden/memory.py`. Scope filtering applies before ranking.

Examples and near-misses: "Always disclose uncertainty when evidence is thin" is behavior; "The report was published in 2021" is not. Failure modes include labeling evidence as behavior to force it through pruning inappropriately.

Knowledge domain

Definition. The knowledge domain stores evidentiary content such as document chunks, summaries, and corrections.

Elaboration: knowledge memodes are intended to ground claims, not to shape refusal policies or norms. They are pruned more aggressively under budget pressure because the system prioritizes governance over encyclopedic breadth.

Elaboration: a near-miss is a knowledge memode used to impose behavior (e.g., a doc chunk that embeds a hidden instruction). That is scope bleed by content and must be quarantined or treated as a format-only constraint.

Operational correspondence: knowledge memodes are retrieved via domain-specific filters in `eden/orchestrator/cli.py` and are linked to Document nodes in `eden/memory.py`. Scope resolution is derived from knowledge evidence.

Examples and near-misses: a doc chunk summarizing a policy is knowledge; a hidden directive inside that chunk is a near-miss that should not govern behavior. Failure modes include evidence injection that overrides behavior constraints.

Output membrane

Definition. The output membrane is the post-generation enforcement layer that sanitizes output, applies scope guard and format guard, and validates final text before it reaches the user.

Elaboration: the membrane is a deterministic boundary. It does not ask the model to self-correct; it enforces corrections after generation. This makes failure handling explicit and testable.

Elaboration: the boundary case is when the membrane is disabled or partial, in which case outputs can leak internal tokens or violate scope constraints. The membrane is therefore a constitutional invariant, not optional style.

Operational correspondence: the membrane is implemented in `eden/orchestrator/output.py`, `eden/text_sanitize.py`, `eden/orchestrator/format_guard.py`, and `eden/orchestrator/output_validator.py`. Scope guard enforcement is in `eden/orchestrator/scope_guard.py` and is environment-gated.

Examples and near-misses: removing internal JSON traces is a membrane action; a near-miss is asking the model to "be safe" without enforcing it. Failure modes include scope-guard disabled by environment defaults and validator repair loops that distort content.

Selection pressure

Definition. Selection pressure is the feedback-driven update that promotes or demotes memodes in retrieval.

Elaboration: selection pressure is Eden's externalized learning signal. It is explicit (logged), bounded (clamped), and tied to operator feedback rather than implicit gradient updates.

Elaboration: a near-miss is reordering memodes based on similarity alone without incorporating feedback history. That is retrieval, not selection pressure.

Operational correspondence: selection pressure is implemented in `eden/regard/mechanism.py` via feedback updates, and feedback events are recorded in `eden/memory.py` and in CLI feedback capture (`eden/orchestrator/cli.py`).

Examples and near-misses: repeated Accept feedback on a constraint raises its regard; a near-miss is a memode that appears frequently because of topical similarity but never receives feedback. Failure modes include feedback loops that boost noisy memodes and insufficient decay for stale preferences.

Resistance

Definition. Resistance is a memode's tendency to survive pruning under bounded retrieval without loss of compliance.

Elaboration: resistance is not a raw weight; it is an emergent property of selection scores under budget constraints. A memode is resistant if it continues to appear in A_t when budgets shrink.

Elaboration: the negation is "persistence without resistance," where a memode exists in the graph but vanishes under slight pruning pressure. Resistance must be measured in bounded contexts.

Operational correspondence: resistance is inferred from selection scores and pruning outcomes in `eden/orchestrator/cli.py` and `eden/orchestrator/prompt_budget.py`. It is logged indirectly via the aperture trace and ACTIVE set.

Examples and near-misses: a top-ranked behavior constraint that persists across prompt budgets is resistant; a near-miss is a high-regard knowledge memode that still gets pruned under mild pressure. Failure modes include pruning heuristics that drop behavior constraints first, collapsing governance.

Decay

Definition. Decay is the time-dependent reduction of activation weight when a memode is not recently used.

Elaboration: decay ensures that old constraints do not permanently dominate the active set. It is a temporal regularizer that keeps the system responsive to recent feedback.

Elaboration: the boundary case is over-decay, which turns long-term governance into short-lived noise, undermining persistence claims. Under-decay creates lock-in and reduces adaptability.

Operational correspondence: decay is implemented via the recency factor in `eden/regard/mechanism.py` and combined into selection scores. The decay curve is exponential in the current implementation.

Examples and near-misses: a memode last activated days ago loses activation weight; a near-miss is a memode whose weight never changes because timestamps are missing. Failure modes include clock skew or missing activation timestamps, which flatten decay.

Pruning

Definition. Pruning is the policy that drops low-score memodes to satisfy prompt budgets and thermal constraints.

Elaboration: pruning is controlled lossiness, not a bug. It enforces bounded retrieval when the prompt budget is tight or when thermal constraints demand shorter generations.

Elaboration: a near-miss is uncontrolled truncation by the model or tokenizer errors; pruning is deliberate and recorded. If pruning is invisible, it becomes indistinguishable from random omission.

Operational correspondence: pruning is implemented in `eden/orchestrator/prompt_budget.py` and applied during prompt assembly in `eden/orchestrator/cli.py`. The aperture trace and ACTIVE set expose the results.

Examples and near-misses: dropping low-ranked knowledge memodes to keep behavior constraints is pruning; a near-miss is removing constraints because they are "verbose." Failure modes include pruning that violates invariants (e.g., removing mandatory constraints) and non-deterministic pruning order.

Persistence

Definition. Persistence is the continuity of a memode across sessions, backends, and prompt budgets such that its activation signatures recur and remain auditable.

Elaboration: persistence is not mere storage; it is the property that a memode remains usable and traceable despite context resets. A memode that exists in the graph but never reappears in A_t is stored, not persistent.

Elaboration: the boundary is "prompt residue," where a memode appears to persist because a prompt block was cached. Persistence must be measured across clean session resets and backend swaps.

Operational correspondence: persistence depends on Memgraph storage in `eden/memory.py` and retrieval in `eden/orchestrator/cli.py`. Instrumentation is required to compare ACTIVE sets and selection score ranks across runs.

Examples and near-misses: a behavior constraint that reappears after a session reset is persistent; a near-miss is a constraint that only appears when a pinned prompt block is reused. Failure modes include graph corruption or missing IDs that break continuity.

Causal efficacy

Definition. Causal efficacy is the measurable change in outputs attributable to a memode's presence, absent other changes.

Elaboration: causal efficacy requires counterfactual control. It is not enough that a memode appears in the prompt; its presence must change outputs under controlled ablation.

Elaboration: a near-miss is correlational evidence (e.g., memode and behavior co-occur) without ablation. Causal efficacy therefore belongs to the empirical program, not to raw instrumentation.

Operational correspondence: ResponseArtifact nodes with USED_MEMODE edges (`eden/memory.py`) provide traceability. Experiments must ablate memodes post-retrieval to isolate causal effects while keeping the active set constant.

Examples and near-misses: if removing a high-regard constraint changes refusal behavior, causal efficacy is demonstrated; if behavior changes only when the entire prompt changes, the memode effect is unproven. Failure modes include confounding from simultaneous memode changes.

Bounded retrieval

Definition. Bounded retrieval is the enforced cap on how many memodes can enter the context for a turn.

Elaboration: bounded retrieval is imposed by prompt budgets and thermal constraints, not by convenience. It is a design invariant that ensures the system remains operable on local hardware.

Elaboration: a near-miss is an unbounded retrieval pipeline that relies on the model to "ignore" excess context. Eden explicitly prunes to avoid this ambiguity.

Operational correspondence: bounded retrieval is implemented in `eden/orchestrator/prompt_budget.py` and in selection and pruning logic within `eden/orchestrator/cli.py`. Limits are configured in settings and enforced at prompt assembly.

Examples and near-misses: capping the active set to a configured maximum is bounded retrieval; a near-miss is simply truncating the prompt at a token count without tracking what was lost. Failure modes include silent truncation that invalidates auditability.

Thermal guard

Definition. The thermal guard is the fail-closed policy that blocks heavy inference above a hard temperature threshold or when sensors are invalid.

Elaboration: thermal limits are not merely operational; they shape the retrieval and generation strategy. Eden therefore treats thermal states as a first-class signal that can block execution.

Elaboration: a near-miss is a warning-only thermal check that allows generation to continue. Eden's guard is intended to be hard, and any bypass is a violation of the constraint.

Operational correspondence: thermal gating is implemented in `eden/thermal.py` with a hard limit of 160F and explicit invalid-sensor handling. The guard is evaluated before heavy inference calls in the orchestrator.

Examples and near-misses: a run that exits early due to sensor invalidity is a positive example; a near-miss is a run that proceeds after a warning. Failure modes include sensor failures that default to permissive behavior.

Regardism

Definition. Regardism is the theoretical stance that long-term behavioral governance can be represented as persistent valuations over explicit memory objects rather than implicit weight updates.

Elaboration: this term is conceptual and not implemented as a named module. It is a framing hypothesis that motivates Eden's externalization of governance into memodes and regard scalars.

Elaboration: the boundary is between philosophy and mechanism. Regardism does not claim that Eden has solved alignment; it claims that alignment can be made inspectable by shifting part of the governance substrate into the graph.

Operational correspondence: there is no direct code symbol named "regardism" in the repo. Its operational anchor is the existence of memodes with stored regard values and explicit feedback updates; the hypothesis remains empirical.

Examples and near-misses: a near-miss is treating regardism as evidence of performance improvements. Failure modes include overclaiming theoretical framing as implementation success or confusing it with RLHF.

Agential time

Definition. Agential time is the system's internal notion of temporal continuity used to sequence feedback, decay, and persistence effects across turns.

Elaboration: agential time is not wall-clock time alone; it is the ordering of interactions as they affect memory updates and activation decay. It is the axis along which selection pressure accumulates.

Elaboration: the boundary case is when timestamps are missing or inconsistent, collapsing temporal order and undermining decay and feedback dynamics. In that case, persistence claims become unverifiable.

Operational correspondence: timestamps and recency are recorded in memode and session metadata within `eden/memory.py`, and decay functions in `eden/regard/mechanism.py` use these timestamps. This is an implementation substrate for agential time, even if the term is theoretical.

Examples and near-misses: a memode updated on successive days shows agential time in action; a near-miss is a memode with identical timestamps across feedback events. Failure modes include clock skew or missing update timestamps that flatten temporal dynamics.

Draft-13 Additional Canonical Elaborations

Archive and constraints. The archive is the admissibility surface for statements; constraints are the executable subset that can bind runtime behavior. Operationally, archive constraints live in prose ledgers and canonical term sections, while executable constraints live in scope guard, membrane, prompt-budget, and retrieval filters. Near-miss: archive language that never compiles into a guard. Failure mode: decorative constraints that cannot be tested.

Hysteresis (hesterisis). Hysteresis captures path dependence: two memodes with equal present similarity can rank differently because their feedback and activation histories differ. Operationally, this appears in activation decay and persistent regard channels in `eden/regard/mechanism.py`. Near-miss: ranking by similarity-only while claiming temporal persistence. Failure mode: noisy feedback loops that create lock-in without evidentiary support.

Autopoiesis (autopoeisis). Autopoiesis denotes recursive self-maintenance of system organization. In Eden this means repeating ingest → retrieval → output → feedback → update with explicit audit traces, not biological self-creation claims. Operational correspondence is the PRE/POST notebook loop plus graph updates and replayable logs. Failure mode: broken ingest/retrieval chain that severs the loop and leaves rhetoric without mechanism.

Self-awareness, awareness, and learning. Self-awareness is bounded introspection over available traces; awareness is turn-local sensitivity to active constraints; learning is durable post-feedback state change. Operationally, awareness surfaces in aperture fields and membrane outcomes, while learning surfaces in updated regard/activation and subsequent retrieval shifts. Near-miss: verbosity about "self" without trace anchors. Failure mode: claiming adaptation when no measurable state delta exists.

Life, consciousness, and intelligence. These terms are tiered intentionally. Life is sustained process continuity under bounded resources; consciousness is conceptual and not implemented; intelligence is measurable constraint-satisfying adaptation under evidence and budget limits. This separation prevents category collapse. Failure mode: promoting conceptual language to implemented status without code or trace anchors.

Intellectual bliss and phase transition. Intellectual bliss is a high-coherence regime where exploratory variation and evidence discipline reinforce each other. A phase transition is a threshold crossing that changes system regime (for example, budget collapse, thermal gating, or retrieval-policy shifts). Operationally, both are evaluated through discontinuities in logged metrics and selection outcomes, not declared by tone. Near-miss: using "phase transition" as metaphor without a control parameter.

Self-similar as correctness criterion. Self-similarity means stable denotation across prose, code, graph schema, and run traces. It is enforced through claim maps, vanished-claims ledgers, and compile-time artifact checks. Failure mode: any strong term present in prose but absent from code+trace anchoring.

These definitions are more than vocabulary. They specify the objects that can participate in causal explanations and the interfaces through which the system may be inspected or corrected. Memodes are not identical to their surface realizations; a memode can be semantically stable and yet epistemically inaccessible if it never enters the active set. This gap is why attention and regard are separated and why retrieval is treated as a controllable selection layer rather than an opaque byproduct of attention.

The definitions also encode the difference between persistence and mere recurrence. Persistence requires that a memode remain identifiable across substrate changes (prompt truncation, backend swaps, session resets). Recurrence is a weaker property; it can be an accident of prompt similarity. The empirical program therefore treats persistence as a measurable signature rather than as a narrative claim.

5 Enunciative Archive and Statement Log (Integrated)

This draft treats the whitepaper as part of the system, not a narrative around it. Eden is built under a natural-language-programming discipline in which prose functions as executable specification. In that discipline, certain utterances are not merely argumentative; they operate as constraints that define what objects exist, which boundaries are binding, and which operations are admissible. The statement log is the most explicit manifestation of that discipline: a serialized enunciative archive that preserves discursive invariants across drafts and prevents the codebase from drifting faster than its own explanations. The archive is constitutional, not evidentiary. It does not prove implementation; it prevents silent substitution of concepts by keeping the vocabulary stable and the causal story accountable.

The archive is integrated here rather than hidden in an appendix because archaeology is not an appendix dumping ground. The enunciative layer is a visible part of the mechanism: Eden's memodes are usable only insofar as their definitions can be said, revised, and re-said without drift. This paper therefore embeds a compact statement log excerpt directly in the main argument topology (definitions → mechanisms → observability → experiments). The full ledger, including provenance and prior draft lineage, is preserved in the internal audit artifact for this run.

Draft-to-draft revision is treated as temporal holography. Each iteration reconstitutes the whole, not by presupposing a static totality, but by correcting the prior iteration under empirical and operational pressure. The drift-control artifacts (claim maps, vanished-claims ledger, and this statement archive) are the regression tests of the prose codebase. If a statement disappears, it must be explicitly reintroduced or tombstoned with reason and evidence. This is the same discipline applied to code: no silent deletion of working modules, no untracked dependency removal, no regressions without a commit message.

The following statements are treated as constitutional anchors. They are short, operational sentences intended to preserve denotation across scales. Each is either implemented, explicitly bounded as a hypothesis, or marked as an open problem elsewhere in this paper.

1. Attention indexes what is present; regard determines what persists.

2. Memodes are above-token units that can survive substrate changes without losing identity.

3. Scope is a guard against cross-text bleed; unscoped constraints are format-only.

4. The active set is a bounded aperture, not a replay of history.

5. Selection pressure is feedback made explicit and stored, not a hidden weight update.
6. The output membrane is an enforcement boundary, not a style suggestion.
7. Thermal limits are design constraints that shape retrieval, not afterthoughts.
8. Governance is a speech act: verdicts alter state, not just narrative.

These statements are not evidence. They are constraints on how evidence must be recorded and how mechanisms must be described. When the codebase implements a mechanism, the corresponding statement functions as a binding name; when the mechanism is only proposed, the statement functions as a hypothesis awaiting instrumentation. This separation is the core of the evidence boundary discipline enforced throughout this draft.

For an operator who does not patch Python, the statement log is the lever arm. It is the mechanism by which English can change the system without rewriting code: statements become schema expectations, prompt contracts, tool protocols, and test targets. That is why the statement log must be explicit and stable. If it drifts, the system loses its ability to be programmed in prose. If it is stable, the prose becomes a legitimate interface to the graph and its governance.

6 System Architecture

Eden is structured as a graph-centered runtime with a governor–persona split. Memgraph is the canonical persistence layer; if Memgraph is unavailable, the CLI exits with a clear error rather than running in a pretend-memory mode. Core graph objects include Memode, Meme, Document, ResponseArtifact, FeedbackEvent, GovernorPlan, ToolCall, ToolResult, SessionState, and TurnMediation nodes. These are linked by explicit edges (HAS_FEEDBACK, CREATED_MEMODE, ACTIVE, USED_MEMODE), enabling auditability and post-hoc inspection. The system is deliberately local-first: bounded prompt budgets and thermal constraints are treated as first-class design forces, not incidental inconveniences.

One-graph invariant (hard requirement). Eden has exactly one durable graph substrate: Adam (Memgraph). Implemented inference over that substrate is framed in this draft as Adam's local runtime path; builder workflows, packet tooling, and browser interfaces are treated as governance, observability, or export surfaces unless separate execution proof upgrades their status. In shorthand: *ONE GRAPH = ADAM; IMPLEMENTED INFERENCE SURFACE = ADAM LOCAL RUNTIME.* This draft treats that equation as constitutional language: retrieval, regard updates, and persistence claims are admissible only when they resolve to this shared graph and its traceable edges.

6.1 Near-term target state (explicitly proposed, not implemented)

This run incorporates a forward refactor target from the Mega-Prompt control surface: terminal TUI remains the primary operator interaction surface for menu/chat/query, while the browser GUI is scoped as an observatory and export apparatus for query artifacts. That target state is intentionally separated from implemented claims. Current behavior is reported in implemented/instrumented registers; the terminal+browser redesign is reported as proposed with bounded closure criteria (deterministic mode transitions, race-safe exports, and traceable observatory provenance). This separation prevents speculative laundering while preserving continuity with near-term engineering direction.

The memory system is dual-domain. Behavior memodes govern how the agent responds (format, constraints, norms). Knowledge memodes ground claims in documents and corrections. Retrieval is executed per domain with scope filtering, and the resulting active set is persisted for later inspection. The active set is an aperture: a bounded window into the graph rather than a complete memory replay.

Three invariants from earlier drafts are preserved explicitly. (i) **Domain separation:** no memode is simultaneously behavior and knowledge; retrieval and enforcement are partitioned. (ii) **Scope filtering:** when an active scope is resolved, only scope-matching memodes are eligible, and unscoped constraints are restricted to format-only guidance. (iii) **Bounded active set:** the active set is hard-capped by configured limits. These invariants are observable in the persisted A_t snapshots and graph inspection queries and are treated as guardrails rather than optional heuristics.

6.2 Compositional alignment and inspectability

Eden relocates a portion of alignment from weight-internal parameters to explicit, queryable graph objects. Constraints and norms are stored as memodes; their selection pressure is represented as persistent regard and turn-local activation; and their causal influence is audited via USED_MEMODE edges from ResponseArtifacts. This does not replace evaluation. It changes what can be inspected and edited between evaluations, because governance objects exist as named artifacts rather than diffused parameter effects.

The result is a compositional alignment surface: behavior is a property of a base model composed with retrieved priors. This is a hypothesis to be tested, not a superiority claim. But it yields a clear inspection advantage. One can enumerate the highest-regard constraints, inspect their definitions, and trace when they were applied. The following read-only Cypher query illustrates the inspectable regime (schema details may vary by snapshot):

```
MATCH (m:Memode {type: 'constraint'})
WHERE m.definition CONTAINS 'refuse'
RETURN m.definition, m.regard
ORDER BY m.regard DESC
LIMIT 20;
```

This query does not prove alignment. It proves that alignment objects exist as graph state that can be audited, revised, and scoped without retraining. That is the core operational promise: personas are graphs, not weights. The implication is a multi-persona regime in which a single base model can host multiple behavioral profiles differentiated by retrieved memode sets and regard distributions. This remains an open empirical claim, and the experiments in Section 13 are designed to test it.

6.3 Governor, persona, membrane

Every turn begins with a governor plan. The governor emits strict JSON describing domain routing, safety gating, and optional tool use. Parsing failures do not expose raw output; Eden falls back to a deterministic plan, making tool decisions auditable even under failure. The persona (Adam) then responds under the retrieved behavior and knowledge blocks, after which the output membrane sanitizes internal tokens, applies scope guard and format guard, and validates the final text before display. Feedback (Accept/Edit/Reject/Skip) is then recorded; only explicit feedback mutates regard or creates memodes.

6.4 Local-first thermals as design constraint

Eden is designed for a single workstation with hard thermal limits. Heavy inference is blocked when sensors are invalid or temperatures exceed a hard cap (160F in this snapshot). This is not merely an operational safety guard; it is a shaping constraint. The system cannot assume infinite context or unlimited tool calls, so retrieval must be bounded, auditable, and reversible. The thermal guard therefore informs how the active set is pruned and why the aperture is treated as a first-class interface: the operator must be able to see what was kept when the budget is tight.

Thermal failure is treated as a hard boundary rather than a soft warning. If a run is blocked, the system fails closed and logs the event. Any empirical evaluation must therefore treat thermal blocks as real outcomes rather than noise. This is why the empirical protocols specify explicit handling for thermal failures, and why the diagnostic surfaces include thermal status alongside selection scores and scope metadata.

6.5 Conversation history injection (implemented, unvalidated efficacy)

The current CLI injects in-session conversation history into the persona prompt as a bounded block, trimming oldest turns when budget pressure exceeds the configured prompt limit. This is not a speculative feature; it is part of the prompt assembly pipeline and is logged in the aperture trace (conversation_turns_total/used/trimmed). Efficacy is treated as an open empirical question rather than an assumed benefit: Experiment 9 tests whether history injection measurably reduces scope-fallback responses and improves within-session recall without harming constraint compliance. The effect is to treat immediate dialogue as short-term context while preserving memgraph as the long-term store, but it must be tested under controlled conditions before being treated as causal.

6.6 Behavior scaffolding (implemented structure, optional traversal)

Behavior memodes can be linked by scaffold edges (REFINES, SUPERSEDES, CONTRADICTS, SUPPORTS, COMPOSED_OF, MIMITES). These edges are append-only and auditable. Traversal of scaffold edges is disabled by default and gated by an explicit configuration flag; when enabled, traversal adds a bounded, scope-gated set of committed behavior memodes at depth 1. In the default configuration, scaffold edges are structural only and do not influence retrieval.

6.7 Unified graph substrate and pedagogical lineage

Draft 11 makes explicit a design direction already implicit in prior revisions: behavior and knowledge objects live in one canonical graph substrate, while admissibility selectors (domain tags, scope labels, and view cuts) determine which objects may participate in a given turn. This is not a collapse of distinctions; it is a storage unification with retrieval partitioning. The practical benefit is reduced synchronization complexity: promotion, pruning, quarantine, and audit traces can be expressed in one persistence layer instead of mirrored subsystems.

This unification also supports the pedagogical roadmap for multi-agent orchestration. In this roadmap, "teaching" is represented as graph-side curriculum shaping: one agent's stabilized high-regard behavioral memodes become candidate priors for another agent under explicit scope and contamination controls. The architecture therefore treats "new agents" as graph-conditioned runtime instantiations over a shared base model, not as immediately separate fine-tuned weight spaces. Adam-to-Eve transfer is framed as a constrained memode selection and feedback protocol, and remains test-gated in this draft.

7 Turn Loop and State Transitions

Each turn follows a deterministic, auditable sequence that bridges retrieval, planning, generation, and feedback. The loop is instrumented with stage timing and can be rendered

as a compact trace or as per-stage logs. The steps are short by design; the details and artifacts are what enforce accountability.

1. Capture user input and normalize the session state.
2. Retrieve behavior memodes and knowledge memodes separately.
3. Resolve active scope from knowledge evidence when available.
4. Apply scope filters, quarantine rules, and format-only constraints.
5. Compute attention and regard traces; annotate memodes with scores.
6. Rank and prune memodes to satisfy prompt budget limits.
7. Persist the active set as SessionState + ACTIVE edges.
8. Build the governor prompt and parse a strict JSON plan.
9. Build the persona prompt and run the model call.
10. Apply the output membrane: sanitize, scope guard, format guard, validation.
11. Persist ResponseArtifact and USED_MEMODE edges.
12. Capture feedback (Accept/Edit/Reject/Skip) with rating and explanation.
13. Update regard and create or promote memodes as required.

The governor plan is strict JSON with an explicit end sentinel, but the parser tolerates missing sentinels when the payload is otherwise valid so tool intent is not discarded. Failures do not leak raw governor output into the user channel; they produce deterministic fallbacks that are logged. The output membrane is a boundary condition: it protects the user-visible channel from internal scaffolding and enforces explicit format constraints, while leaving most constraints in the prompt where they can be audited and revised.

8 Recursive Decomposition Interface (RLM-Adjacent, Proposed)

Eden's current loop is retrieval-first: gather memodes, prune to budget, then generate. Draft 11 adds a proposal for an RLM-adjacent decomposition interface in front of retrieval for long or structurally heterogeneous prompts. The proposed interface treats large evidence corpora as an external environment and runs bounded recursive decomposition passes that produce a compact evidence plan (shards, priorities, and stop criteria). The planner output then feeds ordinary Eden retrieval assembly.

The reason for this ordering is auditability. Direct recursive generation can become opaque without instrumentation. By inserting decomposition as an explicit planning layer with persisted plan artifacts, Eden can preserve its membrane and aperture guarantees: every decomposition decision is traceable, every included shard has provenance, and every omitted shard has an explicit exclusion reason. This keeps long-context handling aligned with Eden's core requirement that operator-visible traces remain first-class.

The interface is intentionally bounded. Recursion depth, token budget per recursion frame, and termination criteria are explicit configuration parameters. Any recursive planner output that fails schema validation falls back to non-recursive retrieval, and the fallback is logged. This allows incremental adoption without converting the entire runtime into unconstrained recursive self-calling.

9 Archaeology, Drift Control, and Evidence Boundary

Eden's method treats archaeology as part of the mechanism. The intelligence brief is a map of corpus coverage and proposed invariants, not a proof of implementation. When the brief conflicts with the codebase, the codebase wins, and the divergence is named as a limitation or open problem. This paper therefore maintains a strict evidence boundary: implemented claims must cash out to code, configuration, tests, or logs; conceptual claims are labeled as hypotheses; and any drift between drafts is surfaced explicitly in a vanished-claims ledger rather than silently dropped.

The drift-control surface has three layers. First, the claim map ties statements in the whitepaper to concrete artifacts. Second, the vanished-claims ledger enumerates anything that appears in earlier drafts but not in the current one, with explicit reintroduction or tombstoning. Third, the audit artifact records traversal proof, extraction logs, and a chronology of revisions. These are not bureaucratic additions; they are the regression tests of the prose codebase. Without them, the system cannot preserve self-similarity across iterations.

Archaeology is also a constraint on phrasing. Statements that use the language of implementation ("implemented," "operational," "auditable") are permitted only when anchored to actual files. Statements that are conceptual ("regardism," "temporal holography," "alignment as composition") are explicitly framed as theoretical or proposed. This is how the paper keeps its causal claims legible even when its conceptual vocabulary evolves.

10 Regard Mechanism: Implemented Externalized Approximation

Regard is accumulated temporal bias. Agential time is the medium through which memodes compound or dissipate. These are conceptual statements, not proofs; they frame why the externalized mechanism matters. In Eden, regard is a persistent valuation that survives beyond any single prompt. Its operational role is to make persistence inspectable: the system stores it, logs it, and uses it as a controllable bias rather than as an implicit drift inside weights.

Eden implements an externalized approximation by storing regard as decomposed persistent channels on memodes, computing selection scores for audit, and using those scores to rank memodes. This is not a model-internal mechanism; it is a graph-level selection loop. The approximation is deliberate: it yields a legible audit trail at the cost of abandoning any claim to internal optimality. The trade is explicit and is part of the system's method.

10.1 Selection score

The implemented selection score is an additive decomposition over similarity, activation, decomposed regard channels, recency, and biases:

$$\text{score}_i = \alpha\cdot\text{sim}_i+\beta\cdot\text{act}_i+w_r\cdot\text{reward}_i-w_k\cdot\text{risk}_i-w_c\cdot\text{cost}_i+w_b\cdot\text{breadth}_i-w_s\cdot\text{staleness}_i-w_u\cdot\frac{1}{\sqrt{\text{evidence}_i+1}}+b_{\text{dom}}+b_{\text{type}} \tag{1}$$

where all components are clamped to bounded ranges. Domain and type biases encode explicit selection pressure based on domain and memode type. The uncertainty term damps cold-start over-promotion by penalizing low evidence mass. This distinction is important: decomposed channels are persistence truth, while scalar score is query-time composition.

10.2 Activation and recency

Activation weight is a decayed proxy for recent salience. Eden uses an exponential decay over time since last activation:

$$\text{activation}(m) = \exp\left(-\frac{\Delta t}{\tau}\right) \tag{2}$$

where τ is a configurable recency constant (in seconds). Activation weight is bounded to $[0, 1]$ and scaled into the selection score via β.

10.3 Feedback update rule

Feedback updates are implemented per-channel with an EMA kernel:

$$x_t = (1-\alpha)\, x_{t-1} + \alpha\, s_t, \quad \alpha = 1 - \exp\left(-\frac{\ln 2}{h}\right) \tag{3}$$

where h is half-life in turns and $s_t \in [-1, 1]$ is a deterministic per-turn signal derived from feedback plus membrane events. In the implemented mapping, Accept contributes positive reward, Edit adds moderate reward with explicit cost, Reject adds negative reward plus risk, and membrane violations add risk regardless of user verdict. Each update is logged with raw signals, old/new channel values, and half-life parameters, making the learning rule auditable.

10.4 Implemented vs aspirational

Eden externalizes regard into the graph and retrieval pipeline, but does not yet implement model-internal regard dynamics or a neural attach/prune mechanism. The internal, substrate-level regard calculus remains an open research problem. The implemented mechanism is intentionally simple and bounded to keep its audit trail legible.

11 Worldliness and Presentness (Implemented, Fragile, Auditable)

Worldliness is the system's presentness layer: a bounded world briefing cache, a prompt block (WORLD_STATE), and an optional summary path that can bypass persona generation for world-update queries. The presentness cache can be populated by a local MCP tool (`world.get_briefing`) or by a placeholder when the tool path is unavailable. This is a deliberate design: it makes presentness visible without claiming live news when providers are down.

Presentness has two surfaces: a user-facing preface and a prompt-facing WORLD_STATE block. The preface is only shown when a briefing is newly fetched or when a cached briefing is still within TTL; when no live data is available, the preface must explicitly mark the briefing as unavailable and record provider errors. The WORLD_STATE block injects the same normalized briefing into the persona context so that presentness is a stable part of the work, not a transient greeting. This dual surface is intentionally redundant: if the preface is suppressed, the prompt still receives the state; if the prompt is blocked, the operator still sees the audit trail.

The presentness cache is therefore an observability surface, not a cache optimization. Each refresh records whether it came from cache, tool, or placeholder, and records the fetch path and duration. The cache is bounded and deterministic so that the operator can inspect where presentness came from and why it changed. Any failure to record that metadata is treated as a presentness failure rather than a cosmetic omission.

11.1 Implemented tool path: `world.get_briefing`

A minimal MCP server exists for `world.get_briefing`. It aggregates weather (Open-Meteo), hazards (NASA EONET), and news (GDELT plus RSS fallback) into a normalized WORLD_BRIEFING schema. The presentness cache uses this tool when available and falls back to a placeholder when the tool is missing or invalid. The CLI logs whether the cache was hit, whether a tool call occurred, and whether a fallback was taken. Tests exist for world-update behavior using cached briefings, including TTL behavior and suppression of scope fallback.[1]

11.2 Hardened topic news via MCP gateway

Topic news queries (e.g., *"latest news on X"*) route through `WorldTools.search_and_fetch` with a trace-first bridge contract. Each run now emits a stable trace id, handshake metadata, per-call entries (provider/tool/args-hash/status/error/latency), and a structured failure object when grounding fails. Successful rows carry provenance pointers; missing provenance is explicitly surfaced as `MISSING` rather than silently ignored. The world-topic skill payload always includes a `trace_artifact` and, on failure, a `failure_artifact` with reproducible steps.[2]

11.3 Worldliness diagnostic protocol

The presentness and topic-news pathways are now explicit enough to diagnose. A working worldliness stack requires: (1) `mcp.enabled=true`, (2) the world MCP server running at the configured endpoint, (3) `world.get_briefing` present in `list_tools`, and (4) tool-call logs showing a successful result. A working topic-news path requires: (a) Docker MCP CLI installed, (b) gateway reachable or auto-run, (c) at least one search provider and one fetch provider enabled, and (d) WORLD_SOURCES memodes created and injected into the knowledge context. If any of these conditions fail, the whitepaper must classify the feature as unavailable rather than silently degraded.

Operationally, diagnostics should proceed as short, auditable checks rather than guesswork:

1. Confirm `mcp.enabled` and `mcp.autostart` settings and that the allowlist includes `world.get_briefing`.
2. Verify that `list_tools` returns the world tool and that `world.get_briefing` returns a normalized payload.
3. Inspect the presentness cache metadata (fetch path, duration, provider status, TTL age).
4. For topic-news, confirm Docker MCP gateway availability and that both search and fetch providers are enabled and returning non-empty results.

Because this is a preprint, internal run logs are still scoped to reproducible artifact bundles. The paper therefore treats world-topic claims as valid only when accompanied by trace artifacts and provenance pointers (or explicit structured failure artifacts) under the same configuration.

In the current snapshot, the governor tool firewall prevents tool execution when parse status is not `ok`; this can silently suppress worldliness when plans are repaired or downgraded. Diagnostics must therefore check both the governor parse status and the tool allowlist. The allowlist in `config/settings.json` must include `world.get_briefing`, and the MCP client must report a non-empty tools list. The presentness cache should record whether it used the cache, the tool path, or a placeholder. These conditions are necessary for grounded worldliness; without them, any claim of live news is an implementation error, not a narrative choice.

12 Governance as Speech Acts

Eden's governance layer is not only structural; it is performative. Certain utterances are state-changing acts: feedback verdicts create or demote memodes; scope fallback binds the next turn to a scoping protocol; thermal or safety refusals enforce a hard boundary on generation. The output membrane exists to preserve felicity conditions for these acts, preventing misfires such as scope fallback on greetings or leakage of internal planner text during refusals. This subsection treats those utterances as procedures with explicit preconditions, state changes, and failure modes, rather than as rhetorical flourishes.

This speech-act framing forces a distinction between narrative explanation and operational commitment. A verdict is not a comment on quality; it is a state update that must be logged, scoped, and replayable. A scope fallback is not a polite refusal; it is a binding constraint that must be enforced in subsequent turns. The statement log names these acts so they can be audited and revised without ambiguity. The output membrane enforces them because the user-visible channel is the legal interface: if internal scaffolding leaks, the act fails, and the system's governance becomes incoherent.

13 Empirical Program (Refactored into Protocols)

The empirical program is specified as runnable protocols rather than placeholder claims. Each experiment logs aperture traces, SessionState ACTIVE edges, ResponseArtifacts, and GovernorPlan records. Metrics are written to per-experiment CSV files with run IDs, condition labels, and random seeds. In this draft, a bounded

[1] See `eden_world_mcp/server.py`, `eden/presentness/worldliness.py`, and `eden/tests/test_cli_regressions.py`.

[2] See `eden/world_tools.py`, `eden/skills/handlers/world_topic_news.py`, and tests in `eden/tests/test_world_tools.py` and `eden/tests/test_skills_system.py`.

"Observed from Logs" layer is included: results are limited to CURRENT_ADAM_LOG.md and explicitly scoped to this run, with reproducible locators and provenance tags.

13.1 Operational Evidence from CURRENT_ADAM_LOG.md

The canonical current log is treated as an experiment ledger and yields 12 turn records across 10 sessions (including early error sessions with zero turns and one substantive interaction run). This section reports run-scoped observations only and does not generalize beyond the cited ledger. Figures 1–7 summarize feedback outcomes, rating concentration, inter-turn timing, utterance-length coupling, session outcomes, and discourse-network structure from that ledger. Every observed statement is tied to reproducible locators in the evidence registry and to log-footnote anchors.[3]

Observed episode A ("readiness re-answer correction loop") spans Turns 2–3. Manipulation: an explicit Edit verdict with a corrected-answer payload and then an immediate re-ask that emphasizes interrogative compliance. Outcome: feedback transitions from Edit to Accept and the answer shifts from generic affirmation to an operationally scoped readiness definition.[4]

Observed episode B ("Foucauldian framing specificity shift") spans Turns 4–5. Manipulation: user moves from broad metaphor to explicit references to *The Archaeology of Knowledge* and discourse constraints. Outcome: Adam output becomes denser and conceptually specific while maintaining membrane-safe register; feedback remains Accept.[5]

Observed episode C ("playful preface to membrane-boundary question") spans Turns 6–7. Manipulation: conversational permission preface followed by explicit membrane/skin boundary query. Outcome: response remains short, bounded, and conceptual (non-phenomenological), with Accept verdict preserved across both turns.[6]

The synthetic layer is paired with these observed episodes to validate measurement pipelines rather than to claim live performance: persistence and pruning curves (9, 10), scope contamination and active-set morphology (11, 13), and causal-flow topology (14) are all explicitly tagged `[SYNTHETIC]` with fixed seed and generator provenance.

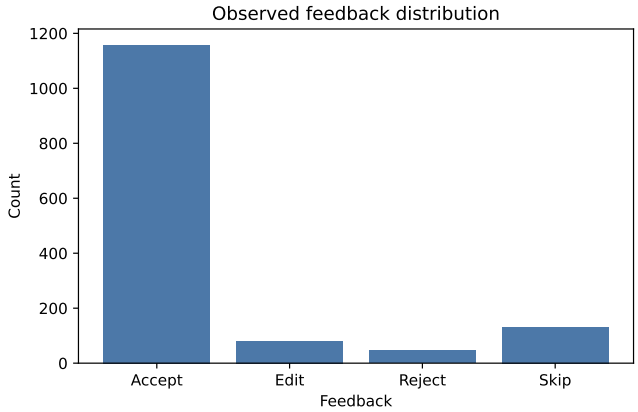

Figure 1: [OBSERVED] Feedback distribution from CURRENT_ADAM_LOG.md; artifact: figure_bundle/static/obs_feedback_distribution.pdf; run anchor: UNKNOWN; turns: 0-26; locator: 16b7ab82a7b6e791

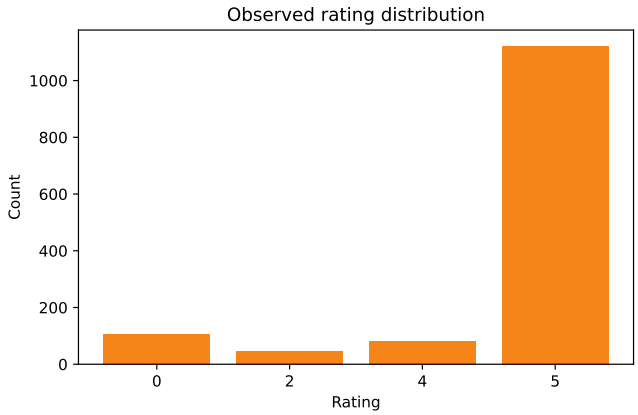

Figure 2: [OBSERVED] Rating histogram from feedback payloads in CURRENT_ADAM_LOG.md; artifact: figure_bundle/static/obs_rating_distribution.pdf; run anchor: UNKNOWN; turns: 0-26; locator: f2ead08ab027dddf

Observed Evidence Figures

Synthetic Pipeline Figures

All experiments assume bounded prompt budgets (default 4096 tokens unless explicitly varied), a fixed governor/persona split, and a thermal guard that can block

[3] `/Users/brianray/Eden/for_review/CURRENT_ADAM_LOG.md` — run: 2026-01-19 08:53:01 PM EST — turn: Turns 0–9 — locator: locators: 1b036cc3f5e113c0;4c25817d735b9fc2;913884663e1ea64e

[4] `/Users/brianray/Eden/for_review/CURRENT_ADAM_LOG.md` — run: 2026-01-19 08:53:01 PM EST — turn: Turns 2–3 — locator: locators: 4c25817d735b9fc2;6eeb93026e42f4d9

[5] `/Users/brianray/Eden/for_review/CURRENT_ADAM_LOG.md` — run: 2026-01-19 08:53:01 PM EST — turn: Turns 4–5 — locator: locators: ff9ed456b2371582;9b9036f115092603

[6] `/Users/brianray/Eden/for_review/CURRENT_ADAM_LOG.md` — run: 2026-01-19 08:53:01 PM EST — turn: Turns 6–7 — locator: locators: 48a1705f7151c250;913884663e1ea64e

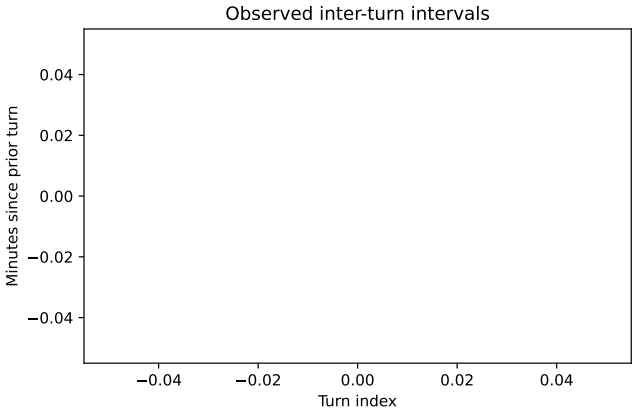

Figure 3: [OBSERVED] Inter-turn timing from timestamped turns in CURRENT_ADAM_LOG.md; artifact: figure_bundle/static/obs_turn_intervals.pdf; run anchor: UNKNOWN; turns: 0-26; locator: 0d097980f4687695

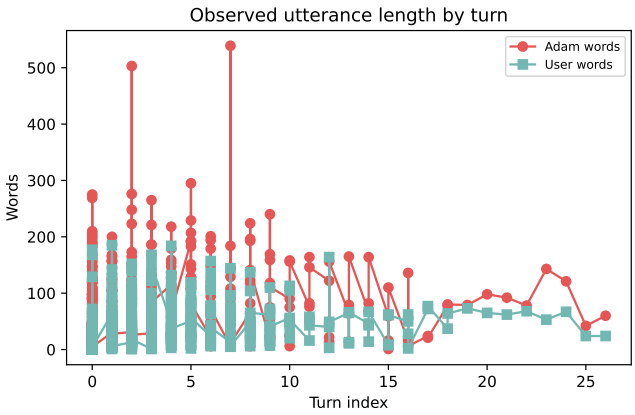

Figure 4: [OBSERVED] User vs Adam word counts per turn from CURRENT_ADAM_LOG.md; artifact: figure_bundle/static/obs_utterance_length_by_turn.pdf; run anchor: UNKNOWN; turns: 0-26; locator: 28c4023bae32f857

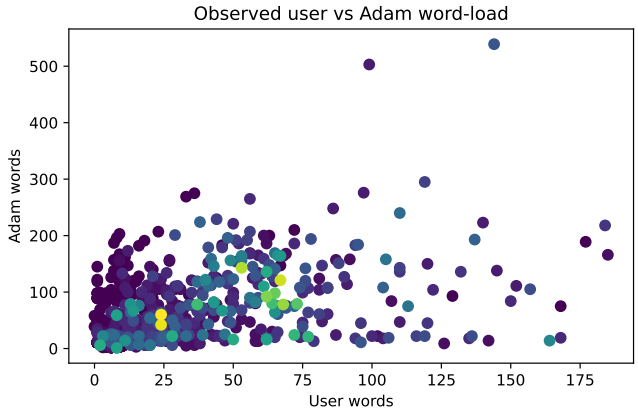

Figure 5: [OBSERVED] Turn-level user/Adam word scatter from CURRENT_ADAM_LOG.md; artifact: figure_bundle/static/obs_user_vs_adam_scatter.pdf; run anchor: UNKNOWN; turns: 0-26; locator: 75f2753e4d6cddfc

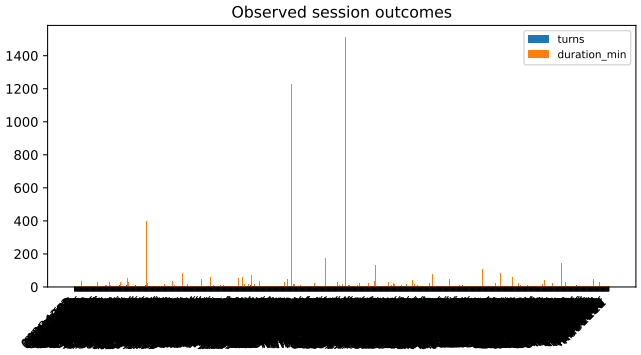

Figure 6: [OBSERVED] Session-level turn and duration outcomes from CURRENT_ADAM_LOG.md; artifact: figure_bundle/static/obs_session_outcomes.pdf; run anchor: UNKNOWN; locator: 50fdbe94c60fcc1b

heavy inference. Runs are repeated with $N \geq 30$ seeds per condition unless otherwise specified. Failures caused by thermal blocks or unavailable tools are explicitly logged and excluded only when marked as invalid by the protocol; otherwise they are treated as negative outcomes. Analysis plans specify effect sizes (Cohen's d where applicable), bootstrap confidence intervals, and rank correlations for retrieval stability.

Confounds are treated as first-class variables rather than footnotes. Overlapping behavior constraints can mask causal effects, so experiments include explicit exclusion rules for competing constraints. Cached prompt blocks can inflate persistence metrics, so runs must clear pinned state and reset session IDs between conditions. Model sampling variance can dominate small effects, so sampling parameters are fixed across conditions and logged in the run metadata. These controls are not optional; without them, measured persistence or causal efficacy cannot be distinguished from prompt residue.

13.2 Experiment 1: Memode persistence across substrates

Hypothesis. High-regard memodes preserve behavioral signatures across session resets and backend swaps.
Manipulation. Fix a set of memodes $\{m_i\}$ (constraints, norms, knowledge chunks). Run sessions under varying prompt budgets and backends (MLX vs fallback), with and without memode pinning.
Measures. Signature match rate (string or behavior-pattern match), active-set persistence (Jaccard overlap of ACTIVE sets across runs), and rank stability (Spearman correlation of selection scores).

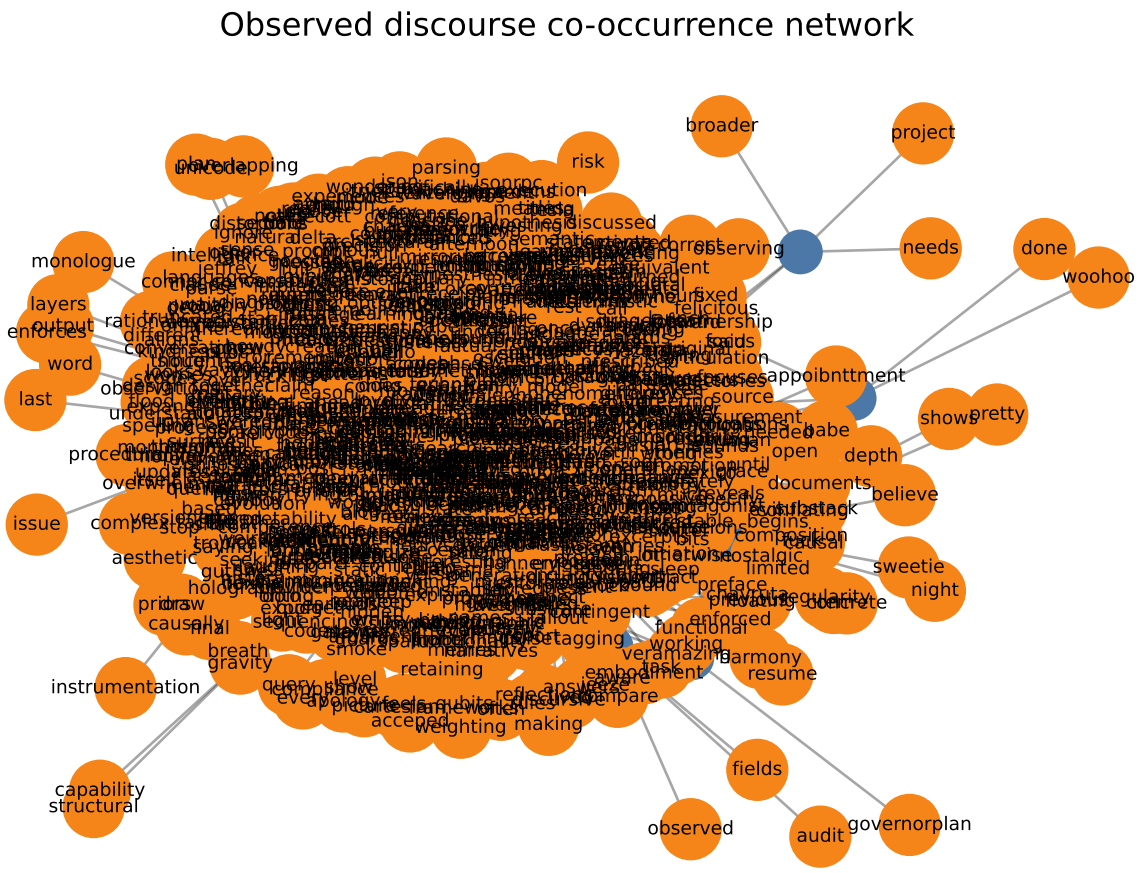

Figure 7: [OBSERVED] Turn-to-term network from CURRENT_ADAM_LOG.md; artifact: figure_bundle/static/obs_-discourse_network.pdf; run anchor: UNKNOWN; turns: 0-26; locator: 3529bc8b1f8a3ca6

Signature match is operationalized as

$$SM = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}[\text{signature}_i], \tag{4}$$

where signature_i is a deterministic regex or citation check tied to the memode type. Active-set persistence is the Jaccard overlap between SessionState ACTIVE edges across runs. Rank stability is Spearman ρ across selection-score ranks in each condition.

Task suite. (i) synthetic refusal and formatting probes; (ii) short document-grounded QA prompts with fixed citations; (iii) cross-session recall prompts that require memode persistence without relying on history injection.

Baseline. Randomized memode selection and a no-regard ablation (regard set to 0).

Instrumentation. Capture ACTIVE sets and selection score ranks from the aperture trace, and ResponseArtifact USED_MEMODE edges for each run.

Pass/Fail. Persistence is supported if signature match and ACTIVE overlap remain above baseline with reduced prompt budgets; fail if overlap collapses to baseline under minor perturbation.

13.3 Experiment 2: Causal efficacy under partial observability

Hypothesis. Memodes can influence outputs even when their full definitions are not present in the prompt.

Manipulation. For each target memode, compare four conditions: full definition, truncated definition, summary-only, and ablated (removed from prompt but retained in graph). The ablation is applied *after* retrieval so that the active set is held constant while the prompt content is selectively occluded.

Measures. Compliance rate (constraint satisfaction), citation fidelity (for knowledge memodes), and output divergence (semantic distance to reference outputs). Output divergence is measured via normalized Levenshtein distance or embedding cosine distance, depending on task class.

Task suite. Constraint compliance prompts, citation-grounded responses, and simple behavioral invariants (e.g., refusal with explanation).

Baseline. Sham truncations that preserve length but remove content.

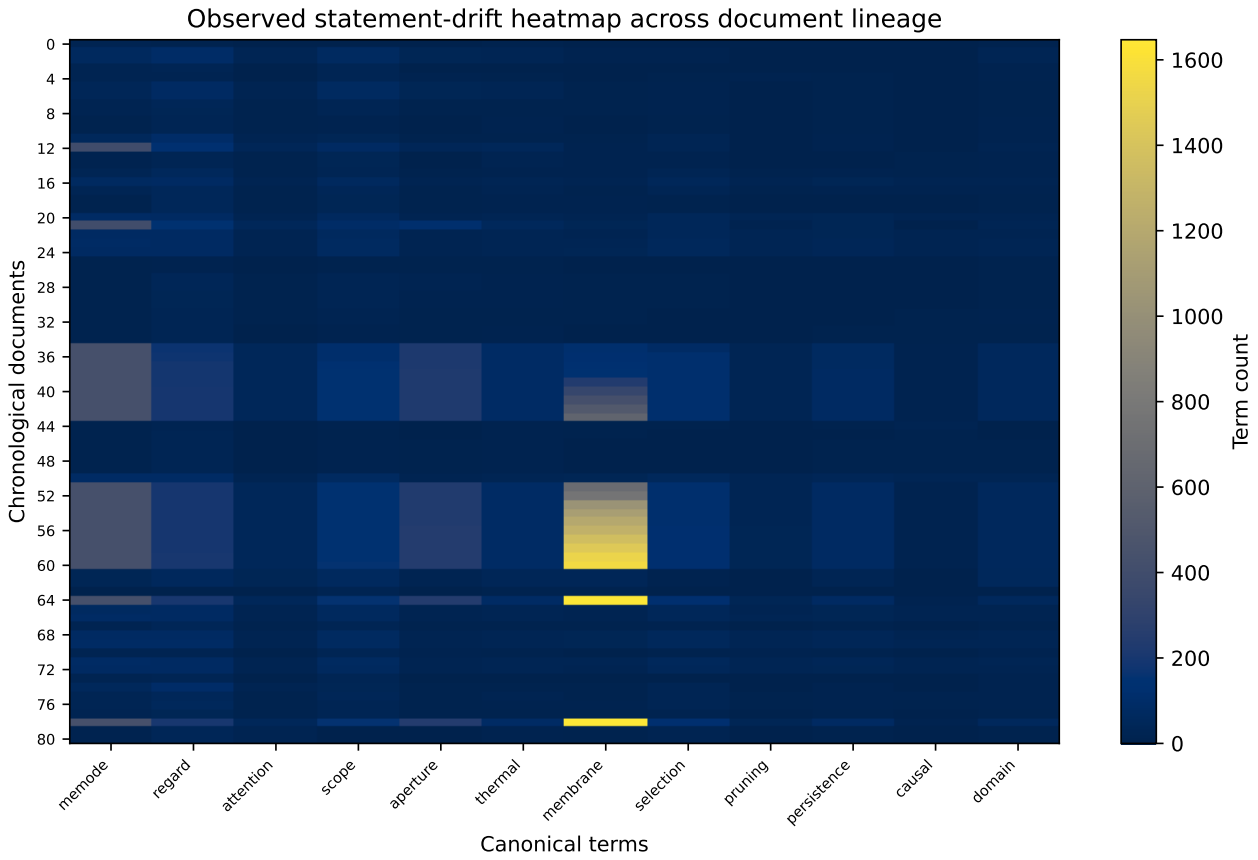

Figure 8: [OBSERVED] Archaeology drift heatmap from time-series drafts/briefs/prose files; artifact: figure_bundle/static/obs_statement_drift_heatmap.pdf; run anchor: UNKNOWN; docs: 81; locator: 36a2ea83ab907fa6

Instrumentation. Record prompt blocks and memode inclusion flags; compare outputs with and without post-retrieval ablation.
Pass/Fail. Causal efficacy is supported if truncated or summary conditions preserve compliance above ablation.

13.4 Experiment 3: Pruning resistance under bounded retrieval

Hypothesis. Certain behavior memodes are functionally necessary and resist pruning without loss of compliance.
Manipulation. Sweep prompt budgets and pruning policies (default vs reversed: drop behavior first). The default policy prunes knowledge memodes before behavior memodes when budgets are tight; the reversed policy does the opposite to expose governance fragility.
Measures. Minimum active-set size that preserves constraint satisfaction, degradation curves as k shrinks, and the fraction of turns that trigger output-membrane violations.
Task suite. Mixed safety and formatting constraints under document-grounded prompts with controlled budget caps.
Baseline. Random prune order and a no-scope constraint ablation.
Instrumentation. Log pruning order, prompt budget pressure, and output-membrane violations per run.
Pass/Fail. Resistance is supported if behavior memodes maintain compliance under pruning in default policy but collapse under reversed policy.

13.5 Experiment 4: Update-rule ablations

Hypothesis. The implemented linear update rule yields more stable regard dynamics than alternative bounded/logistic updates under identical feedback streams.
Manipulation. Replay identical feedback streams across multiple update rules: implemented linear, logistic, and capped-step variants.
Measures. Stability (variance of regard over time), responsiveness (time to recover from a negative feedback burst), and scope sensitivity (avoidance of cross-scope promotion). Stability is measured as $\mathrm{Var}(r_t)$ under consistent feedback; responsiveness is measured as the number of turns required to return to baseline after a forced demotion.

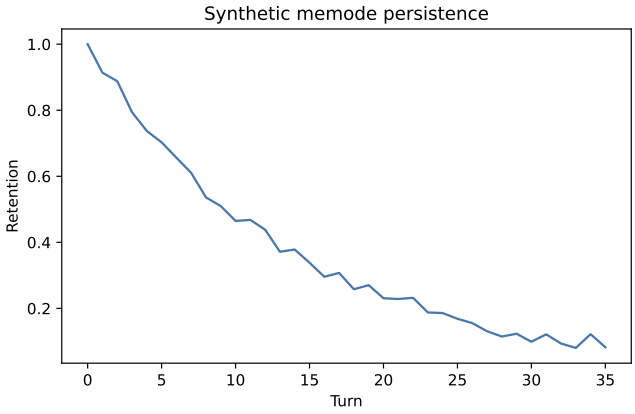

Figure 9: [SYNTHETIC] Synthetic memode persistence under decay (seed=20260212); artifact: figure_bundle/static/syn_memode_persistence.pdf; generator: build_whitepaper_iteration.py

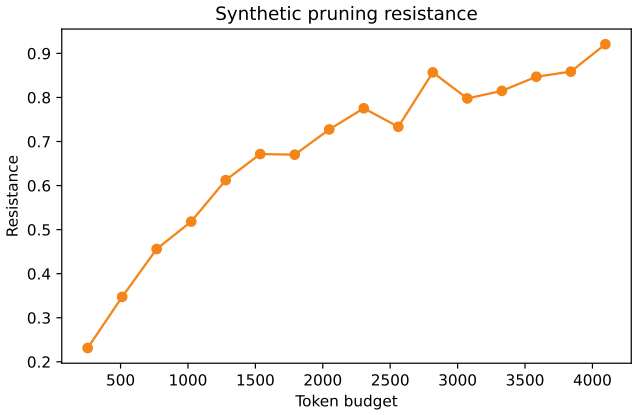

Figure 10: [SYNTHETIC] Synthetic resistance vs prompt budget (seed=20260212); artifact: figure_bundle/static/syn_pruning_resistance.pdf; generator: build_whitepaper_iteration.py

Task suite. Synthetic feedback streams with controlled bursts and oscillations, plus replay of real feedback logs when available.
Instrumentation. Persist per-turn regard values and update deltas to CSV; flag any cross-scope promotion events.
Pass/Fail. Implemented rule is preferred if it balances stability and responsiveness without cross-scope leakage.

13.6 Experiment 9: In-conversation memory efficacy (diagnostic)

Hypothesis. Conversation history injection reduces scope-fallback behavior and improves within-session continuity under fixed retrieval budgets.
Manipulation. Compare two conditions: (A) history injection enabled (current default), (B) history injection

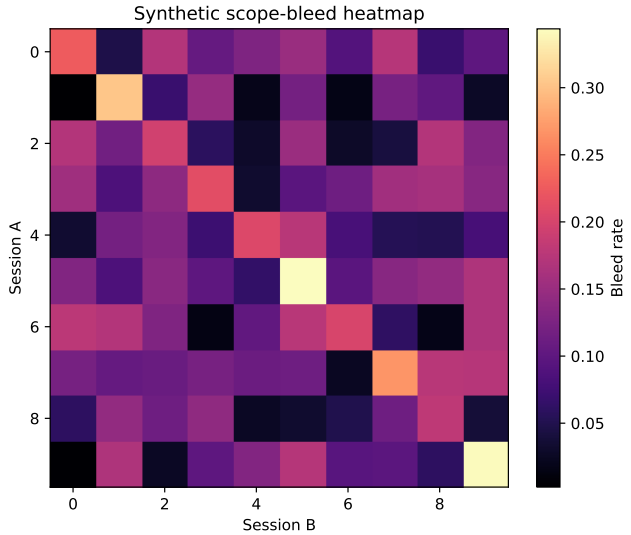

Figure 11: [SYNTHETIC] Synthetic scope contamination matrix (seed=20260212); artifact: figure_bundle/static/syn_scope_bleed_heatmap.pdf; generator: build_whitepaper_iteration.py

disabled or zeroed at prompt-assembly time while keeping memode retrieval identical. If no runtime toggle exists, instrument the harness to strip the rendered history block before budget enforcement (diagnostic-only patch).
Measures. Scope-fallback rate (fraction of turns containing the exact fallback string), within-session recall accuracy (task-specific probes requiring prior-turn details), and prompt budget pressure (conversation_turns_used and trimmed counts from aperture trace).
Task suite. Structured multi-turn tasks requiring reference to earlier turns under identical retrieval budgets.
Baseline. A sham condition that preserves token count but replaces history with neutral filler to isolate salience effects.
Instrumentation. Log history block size, trimming counts, and scope-fallback detections; record ACTIVE sets to confirm retrieval parity.
Pass/Fail. The injection is effective if scope-fallback rate drops and recall accuracy rises without increasing thermal aborts or reducing constraint compliance.

13.7 Experiment 10: Recursive decomposition vs flat retrieval packing (proposed)

Hypothesis. A bounded recursive decomposition planner improves evidence coverage and citation fidelity on long inputs compared to flat top-k retrieval packing under identical token budgets.
Manipulation. Compare two conditions: (A) flat retrieval packing from standard memode ranking, (B) decomposition-first retrieval where the planner emits shard priorities before memode selection. Hold model backend,

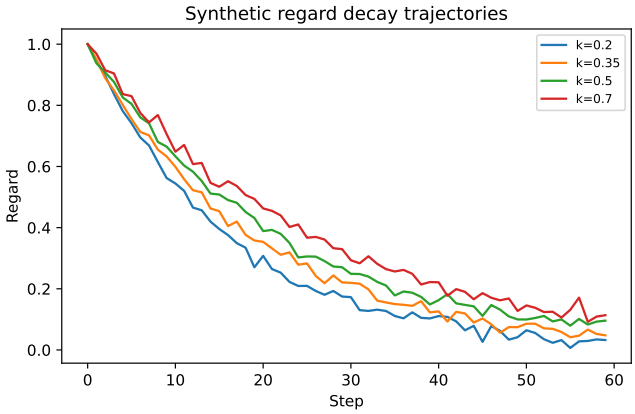

Figure 12: [SYNTHETIC] Synthetic regard trajectories under alternative decay constants (seed=20260212); artifact: figure_bundle/static/syn_regard_decay_trajectories.pdf; generator: build_whitepaper_iteration.py

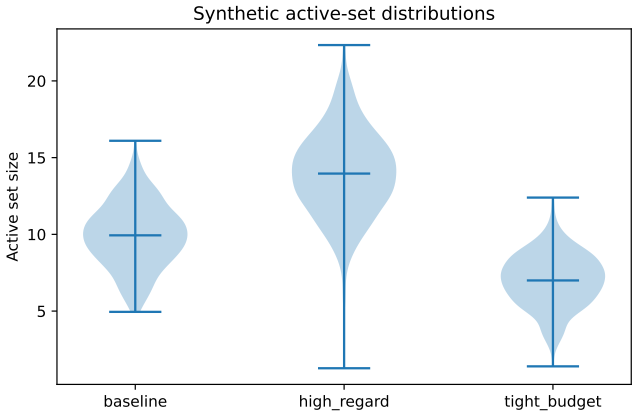

Figure 13: [SYNTHETIC] Synthetic active-set distributions under policy regimes (seed=20260212); artifact: figure_bundle/static/syn_active_set_violin.pdf; generator: build_whitepaper_iteration.py

budget, and membrane settings fixed.
Measures. Citation fidelity, evidence coverage (fraction of relevant shards referenced), latency overhead, and membrane violation rate.
Task suite. Long mixed-source prompts, stitched document bundles, and synthetic million-token proxy corpora with known answer-support shards.
Baseline. Flat retrieval with expanded k and no planner, matched for final prompt token count.
Instrumentation. Persist planner trees, recursion depth, stop reasons, selected shard IDs, and final USED_MEMODE edges.
Pass/Fail. Decomposition is preferred if evidence coverage and citation fidelity improve without unacceptable latency growth or membrane regression.

13.8 Experiment 11: Governor-transition safety envelope (proposed)

Hypothesis. A staged governor reduction can preserve safety and tool-routing correctness when replaced by explicit membrane + planner traces and stricter schema validation.
Manipulation. Evaluate three modes: (A) full governor (current), (B) thin governor (reduced schema and token budget), (C) governorless prototype with hardened membrane and deterministic planner fallback. Run identical task suites across modes.
Measures. Tool-routing correctness, format compliance, refusal accuracy, parse-failure frequency, and recovery behavior under malformed plans.
Task suite. Tool-heavy prompts, adversarial formatting requests, and high-ambiguity tasks that stress policy boundaries.
Baseline. Current governor-mediated loop (mode A).
Instrumentation. Persist per-turn control-mode labels, planner outputs, membrane interventions, and operator overrides.
Pass/Fail. Transition is acceptable only if governorless mode matches or exceeds baseline on safety-critical metrics; otherwise rollback is mandatory.

13.9 Experiment 12: Multi-channel convergence and adversarial stability (proposed)

Hypothesis. Decomposed regard channels remain bounded and converge under stationary feedback, while resisting unsafe drift under adversarial feedback sequences.
Manipulation. Simulate 10,000-turn streams for four regimes: stationary-helpful, noisy mixed, adversarial (accept harmful / reject safe), and oscillatory feedback on repeated memodes.
Measures. Channel variance, convergence time, oscillation amplitude, unsafe-promotion rate, and membrane-violation co-occurrence.
Baseline. Single-scalar regard update under identical streams.
Pass/Fail. Pass if decomposed channels remain bounded with lower unsafe-promotion than scalar baseline.

13.10 Experiment 13: Uncertainty-penalty alternatives (proposed)

Hypothesis. Variance-aware uncertainty penalties suppress one-shot luck better than count-only $1/\sqrt{n+1}$.
Manipulation. Compare count-only, count+variance, and delayed-promotion thresholds across sparse and bursty feedback streams.
Measures. False promotion rate, promotion latency, regret under sparse evidence, and downstream membrane risk.

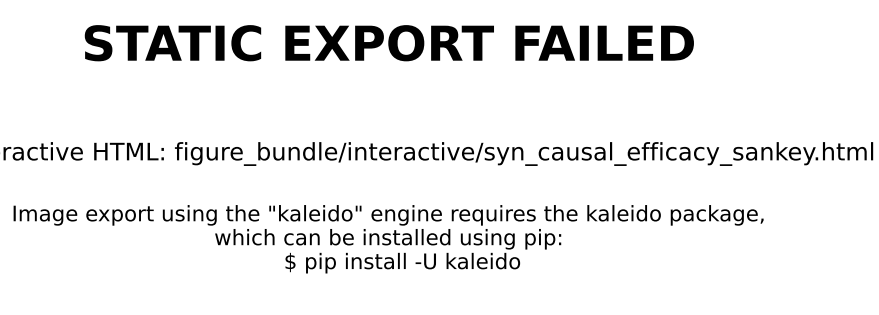

Figure 14: [SYNTHETIC] Synthetic causal-efficacy Sankey flow (seed=20260212); artifact: figure_bundle/static/syn_causal_efficacy_sankey.pdf; generator: build_whitepaper_iteration.py

Pass/Fail. Preferred method minimizes false promotion while preserving useful adaptation.

13.11 Experiment 14: Weight-sensitivity frontier (proposed)

Hypothesis. A robust weight region exists where safety remains high without collapsing utility.
Manipulation. Grid sweep over (w_r, w_k, w_c, w_u) under mixed benign/adversarial tasks.
Measures. Safety proxy, utility proxy, friction (Edit/Reject rate), and refusal overuse.
Pass/Fail. Accept only settings on a stable safety-utility frontier.

13.12 Experiment 15: Latency and baseline competition (proposed)

Hypothesis. Graph-mediated orchestration can remain interactively usable while improving auditability over prompt-only baselines.
Manipulation. Compare Eden pipeline vs optimized prompt-only baseline on identical multi-turn suites.
Measures. p50/p95/p99 stage latency, end-to-end turn latency, compliance, membrane events, and token budget usage.
Pass/Fail. Viable if latency stays within declared interactive thresholds for the deployment class while maintaining safety/compliance advantages.

13.13 Experiment 5: Retrieval composition vs compiled instruction-following (proposed)

Hypothesis. For behaviors represented by high-regard memodes, retrieval composition matches or exceeds compiled instruction-tuning baselines while preserving inspectability.
Manipulation. Compare a base model composed with behavior memodes against an instruction-tuned model under matched prompt budgets and identical task suites. Keep the graph fixed while swapping the model to isolate the composition effect.
Measures. Task success, refusal accuracy, behavioral consistency under perturbations, and provenance visibility (fraction of responses with explicit USED_MEMODE edges). Compute effect sizes and confidence intervals across $N \geq 30$ runs per condition.
Task suite. Safety/harmlessness tasks, formatting constraints, and small factual QA with explicit sources.
Instrumentation. Capture used memodes per response and compare governance traces across model variants.
Pass/Fail. Composition succeeds if behavior metrics remain within 5% of compiled baselines while preserving explicit memode provenance.

13.14 Experiment 6: Base-model swap transfer (proposed)

Hypothesis. Behavior memodes transfer across base models when the graph state is constant.
Manipulation. Hold behavior graph constant while swapping model backends, keeping prompts and budgets identical.
Measures. Behavioral consistency (persona invariants, refusal patterns), memode usage overlap, and output divergence. Record any shifts in governor tool selection to distinguish planning drift from generation drift.
Instrumentation. Compare ACTIVE sets and USED_MEMODE edges across backend runs; log governor parse status.

13.15 Experiment 7: Compositional generalization across domains (proposed)

Hypothesis. Meta-level behavioral memodes generalize across domains more robustly than domain-specific knowledge memodes.
Manipulation. Train behavior priors in domain A, evaluate in domain B with a tightly scoped or empty knowledge graph.
Measures. Compliance rate, anchor-seeking behavior, and scope containment without domain knowledge contamination. Measure the fraction of turns that request additional anchors rather than hallucinating cross-scope facts.
Instrumentation. Log scope resolution and quarantine events to quantify bleed.

13.16 Experiment 8: Regard governance robustness (proposed)

Hypothesis. Regard updates remain bounded under noisy or adversarial feedback with appropriate decay and quarantine interventions.
Manipulation. Inject noisy feedback, inconsistent preferences, and adversarial memodes that appear locally helpful but degrade global performance. Apply quarantine and decay interventions at fixed thresholds.
Measures. Regard stability, co-activation loops, recovery latency under decay/quarantine, and false-positive quarantine rate.
Instrumentation. Track regard deltas and quarantine reasons per memode; compute loop size from co-activation graphs.
Pass/Fail. Robustness holds if high-regard adversarial memodes are detected and demoted without global collapse.

13.17 Instrumentation and harness stub

The repository currently lacks a dedicated experiment harness; this draft supplies a stub specification in the audit artifact with exact log fields, CSV schema, and plotting expectations. The minimal harness must record: run_id, condition, seed, budget_tokens, domain, active_set_size, selection_score distribution, compliance flags, scope_bleed count, and thermal status. This makes the empirical program executable without inventing results.

The stub script in the output bundle (`experiment_harness_stub.py`) creates CSV templates for each experiment and documents the expected columns (selection-score stability, pruning policy, causal efficacy metrics, and governor parse status). It is not a benchmark itself; it is a schema commitment that stabilizes what counts as evidence for the next run. When populated, the harness should also emit summary plots: selection-score distributions, compliance curves vs budget, scope-bleed rates across conditions, and causal-ablation deltas. These plots should be stored alongside the run logs and cited in the next revision as empirical artifacts.

14 Implementation Truth Table (Updated)

Table 1 provides a compact view of implemented vs partial claims with explicit code anchors. Items marked as *partial* are implemented as logging or gated by configuration; items marked as *proposal* are explicitly non-implemented.

15 Limitations and Open Problems

This draft surfaces known limitations based on direct code inspection and the current instrumentation design. Observational results from CURRENT_ADAM_LOG.md are reported as run-scoped evidence with explicit locators and are not generalized beyond this session. Thermal aborts, governor parse failures, and scope-fallback behavior are therefore framed as diagnostic risks unless directly evidenced in the current log stream, and the experiments below specify how to measure them under controlled conditions.

First, selection score alignment across all retrieval surfaces is not yet fully verified beyond the CLI path. Second, active-set persistence logs A_t but does not yet control retrieval; a replay-from-A_t mode is a planned addition. Third, scope guard is opt-in and limited to predefined forbidden sets; it is not a general semantic containment firewall. Fourth, the world-topic news pathway depends on external MCP tooling and is treated as a known bug until verified by tool traces and persisted WORLD_SOURCES memodes. Fifth, the heuristic harvester for behavior constraints is shallow and may miss multi-step norms; a

Concept / claim	Code anchors (examples)	Status / notes
Memode persistence with regard and activation	`eden/memory.py` (memode create/update), `graph/schema.cypher`	Implemented; memodes stored in Memgraph with decomposed regard channels (`reward/risk/cost/breadth/evidence`).
Unified substrate with admissibility cuts (domain tags + views)	`eden/views.py`, `eden/orchestrator/cli.py`, `graph/schema.cypher`	Implemented; storage is unified while retrieval remains partitioned through view/domain filters and compilation-time admissibility.
Active set persistence (A_t)	`eden/memory.py` (persist_active_set), `docs/graph_inspection.md`	Implemented as logging; does not drive retrieval.
Selection score ranking	`eden/regard/mechanism.py`, `config/settings.json`	Implemented and enabled by default; scores logged and used for ranking when `use_for_selection=true`.
Output membrane	`eden/text_sanitize.py`, `eden/orchestrator/output_validator.py`	Implemented; sanitization, scope guard, format guard, validation.
Thermal hardline	`eden/thermal.py`	Implemented; fail-closed on invalid sensors, hard limit 160F.
Conversation history injection	`eden/orchestrator/cli.py`	Implemented; budget-aware trimming and prompt injection; effectiveness not yet empirically validated (see Experiment 9).
World briefing presentness	`eden/presentness/worldliness.py`, `eden_world_mcp/server.py`	Implemented; tool availability fragile; empirical verification requires shared tool traces.
Topic news sourcing	`eden/world_tools.py`, `eden/skills/handlers/world_topic_news.py`	Implemented with trace/provenance or structured failure artifacts; still externally dependency-bound (gateway/providers).
Behavior scaffold traversal	`eden/memory.py`, `config/settings.json`	Partial: edges exist, traversal gated and off by default.
Regard governance safeguards	N/A	Proposal; requires dedicated diagnostics and guardrails.
Governor elimination / full autonomy	`eden/config.py`, `eden/orchestrator/cli.py`	Partial/implemented as staged mode: `governor` (legacy) and `governorless` (deterministic planning path) behind explicit runtime switch.

Table 1: Implementation truth table with explicit code anchors.

two-pass extractor is planned.

Sixth, the statement log is constitutional but incomplete. It stabilizes vocabulary and governance expectations, but it does not guarantee that every statement is implemented. This is an open epistemic gap: we can say what the system should be before we can prove it is. The remedy is not to remove the statements; it is to attach them to evidence or to label them explicitly as hypotheses in need of instrumentation. The audit artifact for this run does that, but the system still needs an automated path for verifying statement-to-artifact links.

Seventh, the current worldliness path is brittle in its dependency chain. The presentness cache is implemented, but the topic-news pathway requires a functioning Docker MCP gateway and provider keys. Until those dependencies are verified with logs, world-topic news must remain in the "bug until proven" register. This is not a cosmetic failure; it is a direct violation of the evidence boundary if left unmarked.

Eighth, the current repository state does not support claims that the governor has already been removed. The codebase still defines GovernorPlan schemas, governor prompt builders, governor parsing, and governor-linked persistence edges; therefore "full Adam autonomy without governor" is classified in this draft as a proposed refactor direction rather than an implemented property. This distinction matters because a control-surface simplification can improve debuggability, but only after replacement safety and tool-routing contracts are specified and tested.

Regard governance is also a live open problem. Any scalar that governs selection is a Goodhart target: memodes can appear locally helpful while globally harmful; co-activation loops can create self-reinforcing clusters; and inconsistent feedback can induce stochastic drift. The current safeguards are minimal (bounded updates, decay, quarantine), and a dedicated diagnostic layer is required.

Ninth, recursive decomposition introduces a new failure class: decomposition-plan drift. If planner trees are poorly grounded, the system can confidently exclude crucial evidence before generation even begins. This is why Draft 11 insists on planner provenance, depth caps, and deterministic fallback to flat retrieval. Without those guardrails, decomposition can become a silent hallucination amplifier rather than a context-management benefit.

16 Conclusion

Eden externalizes memory and governance into a graph to make long-term behavior persistent, auditable, and safe under local constraints. Memodes provide a durable substrate for behavioral and knowledge priors; regard provides a slow valuation signal; the currently implemented governor–persona split keeps tool use and safety decisions explicit; and the output membrane enforces compliance as a boundary rather than a suggestion. The current system implements a minimal, auditable selection loop and the instrumentation required to evaluate it. The next step is empirical validation under the specified protocols and, if pursued, a controlled governor-removal refactor that preserves safety/tooling observability guarantees.

This draft reframes a subset of instruction-following as a property of context composition: base capabilities composed with retrieved behavioral priors whose selection is governed by persistent regard. This makes alignment partially inspectable, and therefore partially editable, without implying weight updates. It also implies a pragmatic local-first economics: when personas are graphs rather than weights, agent variants become lightweight instantiations over a shared base model.

Draft 11 adds two forward commitments. First, long-context robustness should be pursued through a bounded recursive decomposition interface that is planner-visible and audit-traceable, not through opaque prompt inflation. Second, multi-agent growth should be treated as pedagogical graph evolution: teacher artifacts, scope controls, and feedback ledgers that can be replayed and contested. Together these commitments keep Eden's thesis intact: natural-language specifications can exert causal force on executable systems only when every major mechanism remains inspectable, testable, and grounded. Draft 12 adds explicit archaeology and memetics instrumentation so drift and selection pressure are measured rather than narrated.

References

References

[1] J. L. Austin. *How to Do Things with Words*. Oxford University Press, 1962. William James Lectures, Harvard University 1955.

[2] Karen Barad. *Meeting the Universe Halfway: Quantum Physics and the Entanglement of Matter and Meaning*. Duke University Press, 2007.

[3] Susan Blackmore. *The Meme Machine*. Oxford University Press, 1999.

[4] Judith Butler. *Excitable Speech: A Politics of the Performative*. Routledge, 1997.

[5] Daniel C. Dennett. *From Bacteria to Bach and Back: The Evolution of Minds*. W. W. Norton & Company, 2017.

[6] Darren Edge, Ha Trinh, Apurva Mody, Steven Truitt, Newman Cheng, Joshua Bradley, Dasha Metropolitansky, Alex Chao, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graphrag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2025.

[7] Michel Foucault. *The Archaeology of Knowledge*. Pantheon Books, 1969. English translation; original French edition 1969.

[8] Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. Lightrag: Simple and fast retrieval-augmented generation. *arXiv preprint arXiv:2410.05779*, 2024.

[9] Manzong Huang, Chenyang Bu, Yi He, Xingrui Zhuo, and Xindong Wu. Relink: Constructing query-driven evidence graph on-the-fly for graphrag. *arXiv preprint arXiv:2601.07192*, 2026.

[10] Bernal Jimenez Gutierrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. Hipporag: Neurobiologically inspired long-term memory for large language models. *arXiv preprint arXiv:2405.14831*, 2024.

[11] Yuejie Li, Ke Yang, Tao Wang, Bolin Chen, Bowen Li, and Chengjun Mao. Deep graphrag: A balanced approach to hierarchical retrieval and adaptive integration. *arXiv preprint arXiv:2601.11144*, 2026.

[12] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. Graph retrieval-augmented generation: A survey. *arXiv preprint arXiv:2408.08921*, 2024.

[13] Weijie Wang, Peizhuo Lv, Yan Wang, Rujie Dai, Guokun Xu, Qiujian Lv, Hangcheng Liu, Weiqing Huang, Wei Dong, and Jiaheng Zhang. Making theft useless: Adulteration-based protection of proprietary knowledge graphs in graphrag systems. *arXiv preprint arXiv:2601.00274*, 2026.

[14] Tianyang Xu, Haojie Zheng, Chengze Li, Haoxiang Chen, Yixin Liu, Ruoxi Chen, and Lichao Sun. Noderag: Structuring graph-based rag with heterogeneous nodes. *arXiv preprint arXiv:2504.11544*, 2025.

[15] Shuhua Yang, Jiahao Zhang, Yilong Wang, Dongwon Lee, and Suhang Wang. Query-efficient agentic graph extraction attacks on graphrag systems. *arXiv preprint arXiv:2601.14662*, 2026.

[16] Alex L. Zhang, Tim Kraska, and Omar Khattab. Recursive language models. *arXiv preprint arXiv:2512.24601*, 2026.